

Large-Scale and High-Speed FPGA-Based Privacy Amplification for Quantum Key Distribution

Bing-Ze Yan , Qiong Li , Hao-Kun Mao , Hong-Wei Xu, and Ahmed A. Abd El-Latif 

Abstract—In this paper, a high-performance field-programmable gate array (FPGA) based privacy amplification (PA) scheme for both continuous variable and discrete variable - quantum key distribution (QKD) is presented, which can provide more than 1.5 Gbps throughput at the input block size close to 10^8 . Besides, the resource consumption is so economical that there is no need of any external storage. To the best of our knowledge, our scheme outperforms existing FPGA based PA schemes by over an order of magnitude in terms of throughput and input block size. To achieve this feature, we design a novel FPGA-based hybrid hash PA algorithm combining multilinear modular hash and modular arithmetic hash (MMH-MH) based on number theoretical transform (NTT). Based on the hybrid hash algorithm, many optimization strategies are designed in our FPGA based PA scheme, such as an NTT-based optimization unit, an NTT-reusable architecture, a memory-saving modular accumulation unit for multilinear modular hash and a pipelined binary modulo unit for modular arithmetic hash. Plentiful experiment results demonstrate the high performance and reliability of our scheme. It can be embedded in most standalone QKD devices conveniently. What's more, it is also a good candidate for a chip-based QKD system.

Index Terms—FPGA, multilinear modular hash, number theoretical transform, privacy amplification, quantum key distribution.

I. INTRODUCTION

QUANTUM key distribution (QKD) is a notable technique which exploits the principle of quantum mechanics to perform the information theoretical secure key distribution between two remote parties, named Alice and Bob [1]. QKD can be divided into discrete variable (DV) QKD and continuous variable (CV) QKD. A QKD system can be divided into two parts, the quantum optical subsystem and the post-processing subsystem. The quantum optical subsystem is in charge of

preparation, transmission and measurement of quantum states. The post-processing subsystem mainly conducts error correction and privacy amplification [2].

Privacy amplification is a necessary part of the post-processing subsystem in quantum key distribution [3]. It is an art of distilling a highly secure key from a partially secure string by public discussion between two parties. PA requires high throughput to ensure timely processing of input key, and the input block size of PA has significant impact on the final key rate of QKD system due to the finite-size effect [4], [5]. Therefore, throughput and input block size are two key indicators of PA.

Field-programmable gate array (FPGA) is a popular platform to design QKD systems, including PA. It has a high integration density and can handle both the system control task and the calculation task on one chip. Furthermore, the power consumption of a FPGA is much lower than that of other platforms. However, low throughput and small input block size of PA are still the main bottleneck of the FPGA-based QKD system.

Several FPGA-based post-processing subsystems and PA schemes have been designed for high-performance QKD systems [6], [7], [8], [9]. The small input block size is the most critical problem of FPGA-based PA schemes. The largest input block size of existing FPGA-based PA schemes is 10^6 [8], [9], while 10^6 is only the minimum requirement of a common QKD system. The input block size is expected to be much larger than 10^6 to realize a negligible finite size effect on the secure key rate in a high-performance QKD system [10]. The throughput of FPGA-based PA schemes is also insufficient for high-performance QKD system. The highest throughputs of existing FPGA-based PA schemes are 116 Mbps with the block size of 10^6 [9] and 186 Mbps with the block size of 349,952 [8]. However, as the repetition frequency of QKD systems increases and wavelength division multiplexing technology develops, the raw key rate of a QKD system may exceed several gigabits per second and the throughput of a PA scheme is required to the same order of magnitude [11].

In this paper, a large-scale and high-speed (LSHS) FPGA-based privacy amplification (PA) scheme is presented. Its throughput exceeds 1.5 Gbps and its input block size is close to 10^8 . An FPGA-based hybrid hash PA algorithm combining multilinear modular hash and modular arithmetic hash (MMH-MH) is designed for our scheme [12]. This algorithm achieves the best computation complexity $O(n \log n)$ with the optimization method based on number theoretic transform (NTT) [13]. Besides, an NTT-reusable structure and its control unit for our scheme is designed to reduce the resource consumption.

Manuscript received 17 June 2022; revised 19 August 2022; accepted 7 October 2022. Date of publication 14 October 2022; date of current version 1 January 2023. This work was supported by the National Natural Science Foundation of China under Grant 62071151. (Corresponding author: Qiong Li.)

Bing-Ze Yan, Qiong Li, Hao-Kun Mao, and Hong-Wei Xu are with the Faculty of Computing, Harbin Institute of Technology, Harbin, Heilongjiang 150000, China (e-mail: ybz_hit53@163.com; qiongli@hit.edu.cn; mao_hit@qq.com; winaaa@163.com).

Ahmed A. Abd El-Latif is with the EIAS Data Science Lab, College of Computer and Information Sciences, Prince Sultan University, Riyadh 11586, Saudi Arabia, and also with the Department of Mathematics and Computer Science, Faculty of Science, Menoufia University, Al Minufiyah 32511, Egypt (e-mail: a.rahiem@gmail.com).

Color versions of one or more figures in this article are available at <https://doi.org/10.1109/JLT.2022.3214440>.

Digital Object Identifier 10.1109/JLT.2022.3214440

A memory-saving and efficient modular accumulation unit for multilinear modular hash and a pipelined binary modulo unit for modular arithmetic hash are designed to further reduce the resource consumption. Benefit from above optimization strategies, the resource consumption is so economical that there is no need of any external storage. This scheme is implemented on a Xilinx ZYNQ ultrascale+ FPGA, and the performance of our scheme is adequately evaluated. Experiment results indicate that the throughput of this scheme exceeds 1.5 Gbps and the input block size of this scheme is close to 10^8 . To the best of our knowledge, our scheme outperforms existing FPGA-based PA schemes by over an order of magnitude in terms of throughput and input block size.

The rest of this paper is organized as follows. The FPGA-based MMH-MH PA algorithm is designed as a basis for this work in Section II. The structure and optimization strategies of our scheme are designed in Section III. The performance and resource cost of our scheme are evaluated in Section IV.

II. MMH-MH PA ALGORITHM

An FPGA-based MMH-MH PA algorithm is designed in this section. It is the foundation of the LSHS PA scheme.

The main process of a typical PA algorithm is compressing the input sequence with a hash function randomly chosen from a universal hash family [14]. both the multilinear modular hash and the modular arithmetic hash are universal hash families. The multilinear modular hash can not only split the input key and handle it separately but also compress the input key from a binary sequence with length n to a 2^B -nary sequence with length n/B . The computation complexity of multilinear modular hash can be reduced to $O(n \log n)$ by NTT-based optimization [15]. Therefore, the calculation amount of multilinear modular hash is much lower than other universal hash family. However, the output set of the multilinear modular hash is a fixed prime field, which is unfit for PA. Therefore, we designed a new PA algorithm combining the multilinear modular hash and the modular arithmetic hash, which utilizes the variable length bit output set of the modular arithmetic hash.

The MMH-MH PA algorithm performs the compression twice with different universal hash families instead of once. The main steps of MMH-MH PA algorithm are indicated as Fig. 1, where n is the length of input sequence. r is the length of final key. The definition of multilinear modular hash and modular arithmetic hash are given as follow.

a) Definition of Multilinear Modular Hash: Let p be a primer and let k be an integer $k > 0$. Define a family of multilinear modular hash functions from Z_p^k to Z_p as follows:

$$\text{MMH} := \{g_a : Z_p^k \rightarrow Z_p \mid a \in Z_p^k\}, \quad (1)$$

where the function g_a is defined for any $a = \langle a_1, \dots, a_k \rangle$, $x = \langle x_1, \dots, x_k \rangle$, $a_i, x_i \in Z_p$, x means the input variable of the function g_a , and a is a random seed to choose the function g_a ,

$$g_a(x) := a \cdot x \bmod p = \sum_{i=1}^k a_i x_i \bmod p. \quad (2)$$

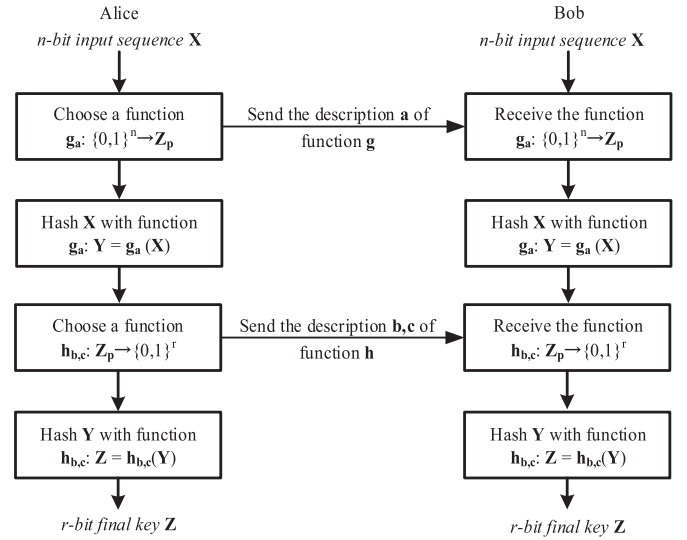


Fig. 1. Main steps of MMH-MH PA algorithm.

MMH family is an universal hash family, its collision probability δ is $1/|Z_p|$, and the proof can be found in [16].

b) Definition of Modular Arithmetic Hash: Let α and β be two positive integers, $\alpha > \beta$. Define a family of modular arithmetic hash functions from 2^α to 2^β as follows:

$$\text{MH} := \{h_{b,c} : Z_{2^\alpha} \rightarrow Z_{2^\beta} \mid b, c \in Z_{2^\alpha}, \gcd(b, 2) = 1\}, \quad (3)$$

where the function $h_{b,c}$ is defined as follows:

$$h_{b,c}(x) := (b \cdot x + c \bmod 2^\alpha) / 2^{\alpha-\beta}. \quad (4)$$

The specific process of the FPGA-based MMH-MH PA algorithm is given as Algorithm 1. We designed a NTT-based optimization to reduce the computation complexity of the MMH-MH PA algorithm to $O(n \log n)$. In details, the prime number p is suggested to be a Mersenne prime. The form of a Mersenne prime is $M_\gamma = 2^\gamma - 1$. The length of input sequence is $n = \gamma \times k$. $x_i = 2^\gamma - 1$ is a special case, and the data $x_i = 2^\gamma - 1$ should be cast away and reload.

Because the process of MMH-MH PA is different from that of traditional PA algorithms, we have proven that the security of MMH-MH PA is similar with other PA algorithms in [17].

III. LARGE-SCALE AND HIGH SPEED PA SCHEME ON FPGA

The LSHS PA scheme on the FPGA is designed based on MMH-MH PA algorithm in this section. NTT is a major part of the FPGA MMH-MH PA algorithm according to Section II. It is the core operation of both MMH function and MH function. Therefore, we designed an NTT-reusable structure and its control unit for the scheme, and we introduced the calculation method of key parameters in this scheme. Afterwards, the design method of main units is illustrated. The most important unit is the NTT-based optimization unit. It deeply determines the performance of the whole scheme. A 64 K NTT processor is implemented to support a maximum 768 Kb input block size. In addition, it can optimize computation resource cost according to the real-time performance requirement by adjusting the radix

Algorithm 1: MMH-MH PA Algorithm.

Input: Input Data: $x \in Z_{2^{\gamma \times k}}$.
 Random Seeds: $a \in Z_p^k, b, c \in Z_{2^\gamma}, \gcd(b, 2) = 1$.
 $//p = M_\gamma = 2^\gamma - 1$

Output: $z \in Z_{2^\beta} // \gamma > \beta$

```

1:  $x = \langle x_1, \dots, x_k \rangle$  //split data  $x$ 
2:  $a = \langle a_1, \dots, a_k \rangle$  //split data  $a$ 
3: if  $x_i = 2^\gamma - 1 (i = 1, \dots, k)$  then
4:   break; //Reload data  $x_i$ 
5: else
6:   for  $i = 0$  to  $k$  do
7:      $y'_i = NTT(x_i) \times NTT(a_i)$ 
8:      $y_i = INTT(y'_i)$ 
9:   end for
10:   $y = \sum_{i=1}^k y_i \bmod p$  /*MMH function:  $y = g_a(x)^*$ 
11:   $z' = NTT(y) \times NTT(b)$ 
12:   $z'' = INTT(z')$ 
13:   $z = (z'' + c \bmod 2^\alpha) / 2^{\alpha-\beta}$ 
    /*MH function:  $z = h_{b,c}(y)^*$ 
14: end if
    
```

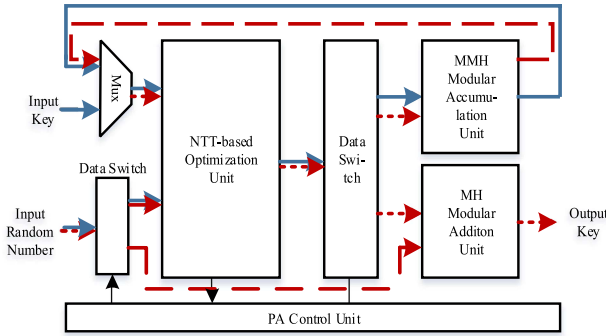


Fig. 2. Architecture of large-scale PA scheme. $\rightarrow$ means the data flow of MMH function and $\rightarrow$ means the data flow of MH function.

of NTT. Then we designed a memory-saving modular accumulation unit for multilinear modular hash and a pipelined binary modular addition unit for modular arithmetic hash to improve overall performance.

The architecture of the LSHS PA scheme is indicated as Fig. 2. The NTT-based optimization unit is reused in this architecture to reduce the resource cost. There are two streams of data flow in this scheme, and they represent the MMH function data stream and MH function data stream. Then we introduced the key parameter calculation and three key units design in the following subsection.

A. Key Parameter Calculation of the LSHS PA Scheme

The sub-block size γ and the sub-block number k are the most critical parameter in this scheme. Because the input block size $n = \gamma \times k$ is the main optimization target, γ and k are expected to be as larger as possible. γ is restricted by two conditions: 1) $2^\gamma - 1$ should be a primer; 2) $2^\gamma - 1$ should be less than

the largest input block size N supported by the NTT-based optimization unit.

The input block size N of the NTT-based optimization unit in our implementation scheme is $2^{786432} - 1$, so the sub-block size γ can be chosen as 756839 ($2^{756839} - 1$ is the 32nd Mersenne prime).

The sub-block number k is restricted by the compression ratio R_{PA} of PA (The calculation method of r can be referred to in [17]), and $1/k$ should be larger than R_{PA} . The compression ratio R_{PA} is affected by the specific QKD system and transmission distance, so the specific value of k will be discussed in next section.

Then we elaborate the design of three main modules of this scheme: 1) the NTT-based optimization unit; 2) the memory-saving modular accumulation unit for multilinear modular hash 3) the pipelined binary modular addition unit for modular arithmetic hash.

B. Design of NTT-Based Optimization Unit

The NTT-based optimization is the most essential and complex unit in this scheme. The input size of the NTT-based optimization unit is 786432 bits. It is implemented based on the radix-16 NTT processor. The NTT-based optimization process can be summarized as follows:

- 1) Break the large numbers X and Y into a sequence of words $x(n)$ and $y(n)$ using base B : $X = \sum x_i \times B^i$ and $Y = \sum y_i \times B^i$.
- 2) Compute the dot product of NTT results $NTT(X)$ and $NTT(Y)$: $Z'_i = NTT(X)_i \times NTT(Y)_i$.
- 3) Compute the inverse NTT (INTT): $Z'' = INTT(Z')$.
- 4) Resolve the carries: let $Z''_{i+1} = Z''_{i+1} + Z''_i / B$, and $Z_i = Z'_i \bmod B$.

The base $B = 24$ and the sequence size $n = 32768$, so the input size of the NTT is $n \times B = 786432$. NTT and INTT are the main parts in this algorithm, and a 65536-point NTT and 65536-point INTT are required. An N-point NTT is defined as:

$$X_k = \sum_{n=0}^{N-1} x_n (W_N)^{nk} \bmod p.$$

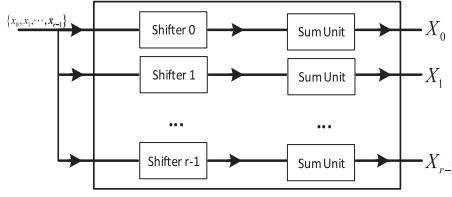
And an N-point INTT is defined as:

$$x_k = N^{-1} \sum_{n=0}^{N-1} X_n (W_N)^{-nk} \bmod p.$$

To simplify the modulo operation, p is chosen as a special primer, $p = 2^{64} - 2^{32} + 1$. One data point in NTT, $NTT(X)_i$, is represented as a 64-bits digit. So the largest data point is $NTT(X)_i \times NTT(Y)_i$, which is a 128-bits digit and represented as $2^{96}a + 2^{64}b + 2^{32}c + d$. It can be rewrite as,

$$\begin{aligned} & 2^{96}a + 2^{64}b + 2^{32}c + d \pmod{p} \\ & \equiv -1(a) + (2^{32} - 1)b + (2^{32})c + d \\ & \equiv (2^{32})(b + c) - a - b + d. \end{aligned}$$

The computation complexity of directly computing NTT is too excessive, so the butterfly algorithm is required to reduce it. The radix is an important parameter of butterfly algorithm.

Fig. 3. Structure of radix- r unit.

Larger radix will decrease the run time of the algorithm and cost more computational resource. The radix- r butterfly algorithm will be introduced next.

1) *Radix- r Butterfly Algorithm*: We take 16-point NTT as an example to demonstrate the difference between radix-2, radix-4, radix-16 butterfly algorithm. The computation of radix-16 is indicated as follows,

$$X_k = \sum_{n=0}^{15} x_n (W_{16})^{nk} \bmod p.$$

Obviously, the radix-16 algorithm only needs to run once to complete the 16-point NTT. The computation of radix-4 is indicated as follows,

$$X_k = \sum_{n=0}^3 x_n (W_4)^{nk} \bmod p.$$

The 16-point NTT can be divided into twice radix-4 calculation, the specific process is shown below,

$$\begin{aligned} X_k &= \sum_{n=0}^{16} x_n (W_{16})^{nk} \bmod p \\ &= \sum_{n_0=0}^3 (W_4)^{n_0 k_1} \left\{ \sum_{n_1=0}^3 x_n (W_4)^{n_1 k_0} \right\} (W_{16})^{n_1 k_1} \bmod p, \end{aligned}$$

where $n = 4n_0 + n_1$ and $k = 4k_0 + k_1$. In the same way, the 16-point NTT can be divided into quartic radix-2 calculation.

It is worth noting that the rotation factor W can be the power of 2 when $p = 2^{64} - 2^{32} + 1$, such as $W_{16} = 4096 = 2^{12}$. Then the multiplication can be replaced by shifting. A radix- r calculation structure can be indicated as Fig. 3. It can be found that the unit with larger radix- r costs more computation resource.

To evaluate the real-time performance of this scheme, we choose the best real-time selection, radix-16.

2) *Structure of the NTT-Based Optimization Unit*: The structure of the NTT-based optimization unit is indicated as Fig. 4. The NTT processor uses a radix-16 unit and matched memory to complete a $16^4 = 65536$ -point NTT/INTT calculation. The calculation requires four stage to complete. The memory unit is divided into 16 banks to load 16-point data in one time. The data in memory should be stored by a well-designed address mapping table. Details of the well-designed address mapping table can be referred to in [13].

The data is load into memory before the NTT calculation. In each stage of NTT, the data is accessed and transmitted into the radix-16 unit. Then it is send to a 64-bit multiplication unit, and the multiplicand will be constant '1', rotation factor W_N^k , INTT factor N^{-1} and the NTT results of NTT_B.

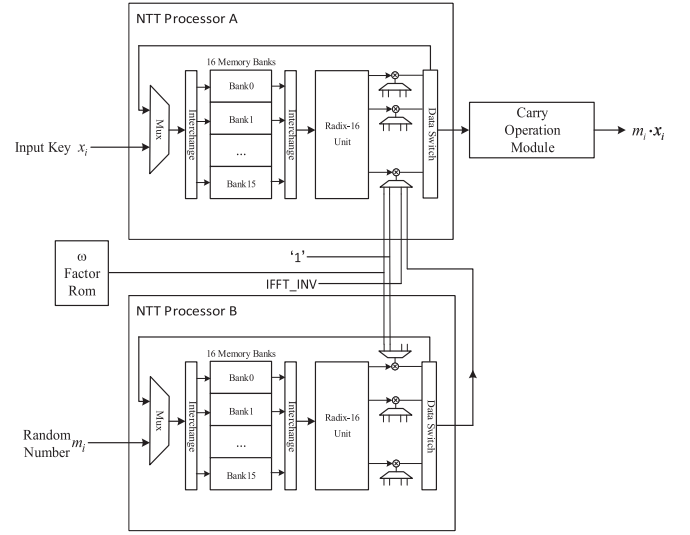
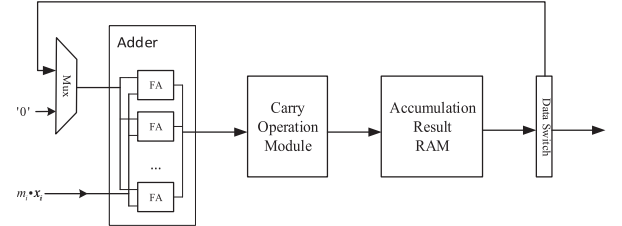
Fig. 4. Structure of the NTT-based optimization unit with $\text{radix} = 16$.

Fig. 5. Structure of memory-saving modular accumulation unit for MMH.

The 65536th primitive root W_{65536} of $p = 2^{64} - 2^{32} + 1$ is $0xed3365469864f124$. After NTT and INTT calculation are completed, the data is load into the carry option module to guarantee each point of results is 24 bits. More details of the NTT processor can be found in [13].

C. Design of Memory-Saving Modular Accumulation Unit for MMH

The low-cost modular accumulation unit is responsible for modular accumulate calculation in MMH function $y = \sum_{i=1}^k y_i \bmod p$, where y_i is the result of the NTT-based optimization unit and $p = 2^{756839} - 1$. The modular addition can be simplified as follows,

$$\begin{aligned} &a + b \bmod (2^{756839} - 1) \\ &= a \bmod 2^{756839} + \lfloor a/2^{756839} \rfloor \\ &\quad + b \bmod 2^{756839} + \lfloor b/2^{756839} \rfloor. \end{aligned}$$

In this way, the modular calculation is replaced by addition and bit operation. It only needs full adders and 756839 bits memory. The structure is indicated as Fig. 5. In this structure, the input data just adds the data in accumulation result memory with period 756839 and memory is cleared when MMH function completes

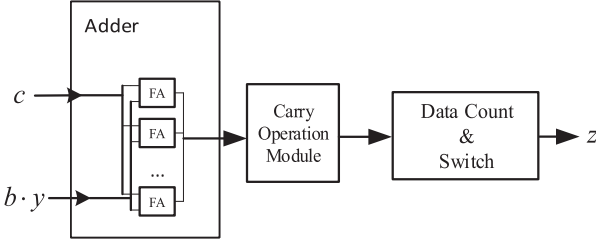


Fig. 6. Structure of pipelined binary modular addition unit.

TABLE I
THE RESOURCE UTILIZATION OF LSHS PA SCHEME

Resource	Scheme Used	Available ¹	Utilization
Luts	156707	230400	67%
BRAMs	198	408	48%
DSP Slices	512	1728	29%

¹ Available: Zynq Ultrascale+ XCZU9EG-2FFVB1156 available

² $\gamma = 756839$ and $radix = 16$

D. Design of Pipelined Binary Modular Addition Unit for MH

The pipelined binary modular addition unit is designed to calculate the equation

$$z = (b \cdot y + c \bmod 2^\alpha) / 2^{\alpha-\beta}. \quad (5)$$

The structure of pipelined binary modular addition unit is indicated as Fig. 6. $b \cdot y$ has been prepared by NTT unit and is the unit input. adder and carry operation module calculate $b \cdot y + c$. The binary modular and division are implemented by the data counter and switch module. The parameter α is equal to γ and β is equal to the length of secure key. Each frame of data is 24 bits, so the module begins to output data when the input data count is $\lfloor (\alpha - \beta) / 24 \rfloor$. The first frame of data outputs $(\alpha - \beta) \bmod 24$ bits data. The rest of frames are 24bits each frame. The output ends when the data count is $\lfloor \alpha / 24 \rfloor$. This unit uses data counter instead of calculation module to implement pipelining and its cost is low.

IV. IMPLEMENTATION AND EXPERIMENT

The proposed LSHS PA scheme is implemented on the Xilinx ZYNQ Ultrascale+ evaluation kit. This kit is based on a Zynq Ultrascale+ XCZU9EG-2FFVB1156 FPGA. The resource utilization is an important indicator to estimate the practicability of a FPGA-based PA scheme, and it is influenced by γ and $radix$ in our scheme. The resource utilization of our scheme with $\gamma = 756839$ and $radix = 16$ is indicated as Table I.

We used the Xilinx Vivado analysis tool for the timing analysis of our FPGA implementation. The clock frequency of this scheme is constrained to 125 MHz. The worst negative slack is 0.614 ns, and the total negative slack is 0.0 ns. The worst hold slack is 0.003 ns, and the total hold slack is 0.0 ns. All timing constraints are met in our scheme.

This scheme is evaluated in three aspects: 1. The input block size of the LSHS PA scheme is evaluated and the secure key rate improvement of a FPGA-based QKD system with LSHS

TABLE II
PARAMETERS OF A TYPICAL DV-QKD SYSTEM AND A TYPICAL CV-QKD SYSTEM

A Typical DV-QKD System[18]	
Parameters	Values
system clock frequency f	1 GHz
transmission distance l	10 km
u, v, w ¹	0.4, 0.1, 0.0007
p_u, p_v, p_w ²	96.9%, 1.6%, 1.4%
η_δ ³	22.5%
SPD dark count rate Y_0	$4.5 \cdot 10^{-6}$
SPD system error e_δ	3%
p_Z, p_X ⁴	96.7% 3.4%
security parameter of PA ϵ	10^{-10}
A Typical CV-QKD System[19]	
Parameters	Values
transmission distance l	50 km
excess channel noise V_e	0.005
electric noise V_{el}	0.041
η ⁵	0.606
reconciliation efficiency β	0.95
security parameter of PA ϵ	10^{-10}

¹ u, v, w : photon fluxes for signal, decoy and vacuum pulses

² p_u, p_v, p_w : respective probabilities for signal, decoy and vacuum pulses

³ η_δ : photon detector (SPD) detection efficiency

⁴ p_Z, p_X : Z and X basis probabilities

⁵ η : homodyne detector efficiency

PA scheme is demonstrated; 2. The throughput of the LSHS PA scheme on different block size is evaluated and compared with existing PA schemes; 3. The resource cost and core indicators of the LSHS PA scheme are compared with existing FPGA-based PA schemes.

A. The Input Block-Size and Secure Key Rate of the LSHS PA Scheme

The input block size n of the LSHS PA scheme is equal to $k \times \gamma$. γ is a fixed value and k is related to the maximum compression ratio R_{PA}^{MAX} of PA. Therefore, the input block size n is related to the maximum compression ratio R_{PA}^{MAX} of PA. R_{PA} can be calculated on the basis of QKD system parameters. The compression ratio of a DV-QKD system can be calculated by $R_{PA} = \beta I_{AB} - I_{AE}(e_1 + \Delta_n)$. The compression ratio of a CV-QKD system can be calculated by $R_{PA} = \beta I_{AB} - \chi_{BE} - \Delta_n$. The main fluctuate factor of compression ratio is the channel error rate e . The maximum compression ratio R_{PA}^{MAX} means the compression ratio R_{PA} when the channel error rate e is minimum. The parameter k is required to be smaller than $1/R_{PA}^{MAX}$ to maximum the secure key rate.

The effect of the LSHS PA scheme on the input block size and secure key rate is simulated by the MATLAB with the parameters of a typical DV-QKD system [18] and a typical CV-QKD system [19] as shown in Table II. The results demonstrate the parameter k , the input block size and secure key rate as communication distance changes in Fig. 7. We computed the compression ratio of PA R_{PA} in two QKD systems at different transmission distances to confirm the parameter k in the first subgraph. Here we assumed the input block size was infinite to

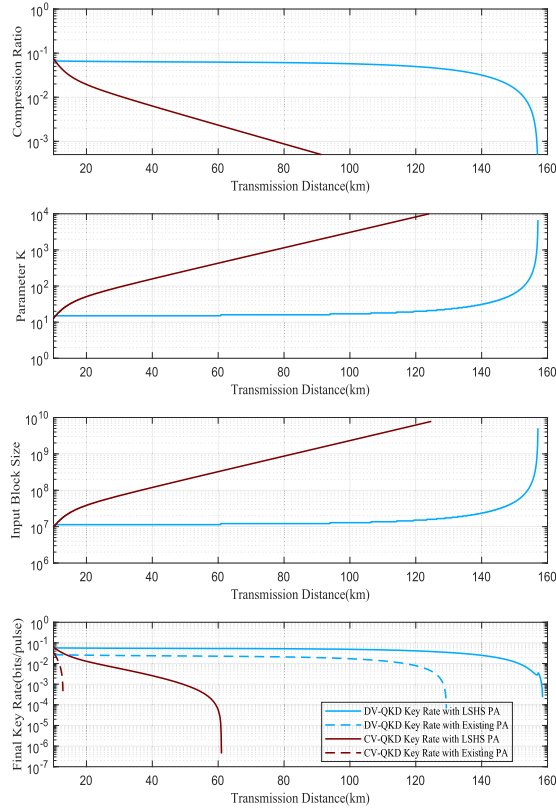


Fig. 7. Final key rate of a QKD system with LSHS PA.

compute the maximum compression ratio in the infinite input block size case. The parameter k of the LSHS PA scheme was calculated according to the compression ratio of PA R_{PA} in the second subgraph. Then the input block size of the LSHS PA scheme at different transmission distances was confirmed with the parameter k and the parameter $\gamma = 756839$ in the third subgraph, where the input block size N equals to $k \times \gamma$. Finally, we simulated the final key rate of typical QKD systems with the LSHS PA scheme at different transmission distances in the last subgraph. The input block size of existing FPGA-based PA schemes is set to 10^6 , because it is the largest input block size of existing FPGA-based PA schemes as we know. It can be found that the effect of the LSHS PA scheme is more significant in the CV-QKD system. This is because the finite size effect in the CV-QKD system is more serious than that in the DV-QKD system.

In conclusion, the simulation results indicate that the LSHS PA scheme can improve the input block size and the secure key rate of a QKD system. This improvement is more efficient in a CV-QKD system.

B. The Throughput of the LSHS PA Scheme

The throughput of PA means the maximum rate of the input key into a PA scheme. It is an important index of PA, which affects real-time of a QKD system. We experimented the throughput of the LSHS PA scheme at different input block sizes. The input key and random number is generated from the

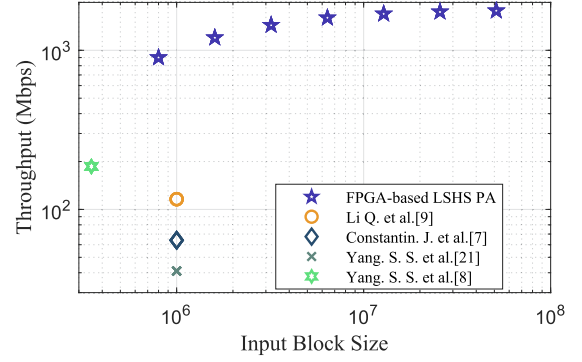


Fig. 8. The throughput comparison of existing FPGA-based PA schemes.

TABLE III
THE RESOURCE COST AND CORE INDEX COMPARISON OF FPGA-BASED PA SCHEMES

	Yang[21]	Constantin[7]	Li[9]	LSHS
Devices	Virtex-7	Virtex-6	Virtex-7	Znyq Ultrascale+
LUTs	15,604	26,571	37,203	156,707
Internal-RAM	100Kb	0Kb	5,652Kb	11,232Kb
External-RAM	0Kb	1,095Kb	128Mb	0Kb
Throughput	64Mbps	41Mbps	116Mbps	1,500Mbps
Block Size	1Mb	1Mb	1Mb	1-100Mb

truly quantum random number generator. Then we compared the throughput between our scheme and existing schemes as indicated in Fig. 8.

The comparison shows that our scheme improves the throughput by an order of magnitude than existing FPGA-based PA schemes, and our scheme reaches a equal throughput with the existing best throughput PA scheme – a GPU-based PA scheme [20]. The throughput advantage of the LSHS PA scheme profits from two reasons: 1. the less computation of MMH-MH PA algorithm; 2. the three efficient units we design in Section III.

C. The Resource Cost and Core Index Comparison of FPGA-Based PA Schemes

We compared the resource cost and core indexes between LSHS PA scheme and existing FPGA-based PA schemes. The resource cost of a FPGA-based PA scheme includes look-up tables (LUTs), internal Random Access Memory (RAM), external RAM and DSP slices. The core indexes to be compared are throughput and input block size of a PA scheme. The comparison results are indicated as Table III. The devices of these schemes are all Xilinx's FPGAs and have similar computation resources, including similar LUTs, RAM and DSP slices.

The LSHS PA scheme costs the most LUTs and Internal-RAM among existing PA schemes. The main cost of LSHS PA is the NTT-based optimization unit. It costs nearly 90% resources of the LSHS PA scheme. The main reason of its high cost is that it chooses the radix of the unit as 16. This provides the best real-time performance and throughput, but also costs the most resources. If an NTT processor with $radix = 2$ was used in this scheme, we estimated that the throughput and the LUTs

cost could be reduced to approximately one eighth of their previous. The internal-RAM cost of the LSHS scheme is the most, because it uses the NTT algorithm for acceleration, and the NTT algorithm needs to store all the input data and rotation factors. Similarly, the scheme in [9] used the FFT algorithm for acceleration. The NTT algorithm has an advantage over FFT algorithm, because the data format of the NTT is integer instead of the floating-point of the FFT. This advantage significantly reduces the memory cost of the LSHS scheme compared with the scheme in [9], and frees the LSHS PA scheme from dependence on the external-RAM.

Although the LSHS PA scheme costs more resources, the LSHS PA scheme greatly improves the core indexes of FPGA-based PA schemes. Before the LSHS PA scheme, the throughput and input block size of the FPGA-based PA schemes are far below those of PA schemes based on other platforms. Therefore, although FPGA-based PA schemes have advantages of low power consumption and high integration level for QKD systems, they are not applied widely due to the low core indexes. The core indicators of the LSHS PA scheme have exceeded existing FPGA-based PA schemes by several orders of magnitude, and our scheme has surpassed the best PA scheme in term of these core indexes.

When we use the LSHS PA scheme to implement the whole QKD post-processing system on one FPGA chip, we should add bit error correction, sifting and authentication cores. The resource cost of sifting and authentication cores is negligibly small. The error correction is the largest resource cost core except PA core [8]. The remaining space on the FPGA chip is enough to build an integrated QKD post-processing system.

V. CONCLUSION

In this work, a large scale and high-speed PA scheme based on FPGA is proposed to improve the input block size and throughput of an FPGA-based PA scheme. A novel FPGA-based MMH-MH PA algorithm based on NTT is designed for this scheme. This scheme is implemented on a Xilinx UltraScale+ FPGA. The throughput of this scheme can reach more than 1.5 Gbps. The input block size of this scheme can be close to 10^8 . The results of throughput and input block size outperform existing FPGA based PA schemes by over an order of magnitude. Besides, the resource consumption is so economical that there is no need of any external storage.

As the rapid development of chip-based QKD systems, the demand on integration and power consumption of the post-processing subsystem constantly emerges. This high-performance FPGA-based PA scheme is a competitive solution for the chip-based QKD system.

In the future, we will design an integrated QKD system based on the LSHS PA core on one FPGA chip to improve its practicability in a real QKD system.

REFERENCES

- [1] C. H. Bennett and G. Brassard, "Quantum cryptography: Public key distribution and coin tossing," *Theor. Comput. Sci.*, vol. 560, pp. 175–179, 1984.
- [2] H. Mao, Q. Li, Q. Han, and H. Guo, "High throughput and low cost LDPC reconciliation for quantum key distribution," *Quantum Inf. Process.*, vol. 18, 2019, Art. no. 232. [Online]. Available: <http://arxiv.org/abs/1903.10107>
- [3] C. H. Bennett, G. Brassard, C. Crkpeau, and U. M. Maurer, "Generalized privacy amplification," *IEEE Trans. Inf. Theory*, vol. 41, no. 6, pp. 1915–1923, Nov. 1995. [Online]. Available: https://ieeexplore.ieee.org/xpls/abs_all.jsp?arnumber=476316
- [4] F. Furrer et al., "Continuous variable quantum key distribution: Finite-key analysis of composable security against coherent attacks," *Phys. Rev. Lett.*, vol. 109, no. 10, pp. 1–5, 2012.
- [5] M. Tomamichel, C. C. W. Lim, N. Gisin, and R. Renner, "Tight finite-key analysis for quantum cryptography," *Nature Commun.*, vol. 3, 2012, Art. no. 634.
- [6] H. F. Zhang et al., "A real-time QKD system based on FPGA," *J. Lightw. Technol.*, vol. 30, no. 20, pp. 3226–3234, 2012.
- [7] J. Constantin et al., "An FPGA-Based 4 mbps secret key distillation engine for quantum key distribution systems," *J. Signal Process. Syst.*, vol. 86, no. 1, pp. 1–15, 2017. [Online]. Available: <http://dx.doi.org/10.1007/s11265-015-1086-1>
- [8] S. S. Yang, Z. G. Lu, and Y. M. Li, "High-speed post-processing in continuous-variable quantum key distribution based on FPGA implementation," *J. Lightw. Technol.*, vol. 38, no. 15, pp. 3935–3941, 2020.
- [9] Q. Li, B. Z. Yan, H. K. Mao, X. F. Xue, Q. Han, and H. Guo, "High-speed and adaptive FPGA-Based privacy amplification in quantum key distribution," *IEEE Access*, vol. 7, pp. 21482–21490, 2019, doi: [10.1109/ACCESS.2019.2896259](https://doi.org/10.1109/ACCESS.2019.2896259).
- [10] D. Rusca, A. Boaron, F. Grünenfelder, A. Martin, and H. Zbinden, "Finite-key analysis for the 1-decoy state QKD protocol," *Appl. Phys. Lett.*, vol. 112, no. 17, 2018, Art. no. 171104. [Online]. Available: <http://dx.doi.org/10.1063/1.5023340>
- [11] T. A. Eriksson et al., "Wavelength division multiplexing of 194 continuous variable quantum key distribution channels," *J. Lightw. Technol.*, vol. 38, no. 8, pp. 2214–2218, 2020.
- [12] B. Yan, Q. Li, H. Mao, and N. Chen, "An efficient hybrid hash based privacy amplification algorithm for quantum key distribution," *Quantum Inf. Process.*, vol. 21, no. 4, pp. 1–15, 2022.
- [13] W. Wang, X. Huang, N. Emmart, and C. Weems, "VLSI design of a large-number multiplier for fully homomorphic encryption," *IEEE Trans. Very Large Scale Integr. Syst.*, vol. 22, no. 9, pp. 1879–1887, Sep. 2014.
- [14] J. L. J. Carter and M. M. N. M. Wegman, "Classes of hash functions," *J. Comput. Syst. Sci.*, vol. 18, pp. 143–154, 1979.
- [15] J. M. Pollard, "The fast fourier transform in a finite field," *Math. Comput.*, vol. 25, no. 114, pp. 365–365, 1971.
- [16] S. Halevi and H. Krawczyk, "MMH: Software message authentication in the gbit/second rates," *Lecture Notes Comput. Sci. (including subseries Lecture Notes Artif. Intell. Lecture Notes Bioinf.)*, vol. 1267, pp. 172–189, 1997.
- [17] Y. Bingze, L. Qiong, M. Haokun, and C. Nan, "An efficient hybrid hash based privacy amplification algorithm for quantum key distribution," 2021. [Online]. Available: <http://arxiv.org/abs/2105.13678>
- [18] Z. L. Yuan et al., "10 mb/s quantum key distribution," *J. Lightw. Technology*, vol. 8724, no. C, pp. 1–7, 2018.
- [19] Y. Zhang et al., "Continuous-variable QKD over 50 km commercial fiber," *Quantum Sci. Technol.*, vol. 4, no. 3, 2019, Art. no. 035006.
- [20] X. Wang, Y. Zhang, S. Yu, and H. Guo, "High-speed implementation of length-compatible privacy amplification in continuous-variable quantum key distribution," *IEEE Photon. J.*, vol. 10, no. 3, pp. 1–9, Jun. 2018.
- [21] S. S. Yang, Z. L. Bai, X. Y. Wang, and Y. M. Li, "FPGA-Based implementation of size-adaptive privacy amplification in quantum key distribution," *IEEE Photon. J.*, vol. 9, no. 6, pp. 1–8, Dec. 2017.